

1960s. Computers were all text-based until Doug Engelbart invented the mouse and the graphical user interface at Stanford Research Institute in the late 1960s. Who knows what will come next?

In this section, and in comparable sections throughout the book, we will take a brief look at some of the research in operating systems that has taken place during the past 5–10 years, just to give a flavor of what might be on the horizon. This introduction is certainly not comprehensive. It is based largely on papers that have been published in the top research conferences because these ideas have at least survived a rigorous peer review process in order to get published. Note that in computer science—in contrast to other scientific fields—most research is published in conferences, not in journals. Most of the papers cited in the research sections were published by either ACM, the IEEE Computer Society, or USENIX and are available over the Internet to (student) members of these organizations. For more information about these organizations and their digital libraries, see

ACM	http://www.acm.org
IEEE Computer Society	http://www.computer.org
USENIX	http://www.usenix.org

All operating systems researchers realize that current operating systems are massive, inflexible, unreliable, insecure, and loaded with bugs, certain ones more than others (*names withheld to protect the guilty*). Consequently, there is a lot of research on how to build better ones. Work has recently been published about bugs and debugging (Kasikci et al., 2017; Pina et al., 2019; Li et al., 2019), crashes and recovery (Chen et al., 2017; and Bhat et al., 2021), energy management (Petrucci and Loques, 2012; Shen et al., 2013; and Li et al., 2020), file and storage systems (Zhang et al., 2013a; Chen et al., 2017; Maneas et al., 2020; Ji et al., 2021; and Miller et al., 2021), high-performance I/O (Rizzo, 2012; Li et al., 2013a; and Li et al., 2017), hyperthreading and multithreading (Li et al., 2019), dynamic updates (Pina et al., 2019), managing GPUs (Volos et al., 2018), memory management (Jantz et al., 2013; and Jeong et al., 2013), embedded systems (Levy et al., 2017), operating system correctness and reliability (Klein et al., 2009; and Chen et al., 2017), operating system reliability (Chen et al., 2017; Chajed et al., 2019; and Zou et al., 2019), security (Oliverio et al., 2017; Konoth et al., 2018; Osterlund et al., 2019; Duta et al. 2021), virtualization and containers (Tack Lim et al., 2017; Manco et al., 2017; and Tarasov et al., 2013) among many other topics.

1.10 OUTLINE OF THE REST OF THIS BOOK

We have now completed our introduction and bird’s-eye view of the operating system. It is time to get down to the details. As mentioned, from the programmer’s point of view, the primary purpose of an operating system is to provide some

key abstractions, the most important of which are processes and threads, address spaces, and files. Accordingly the next three chapters are devoted to these topics.

Chapter 2 is about processes and threads. It discusses their properties and how they communicate with one another. It also gives a number of detailed examples of how interprocess communication works and how to avoid some of the pitfalls.

In Chap. 3 we study address spaces and their adjunct, memory management. The important topic of virtual memory will be examined, together with paging.

Then, in Chap. 4, we come to the topic of file systems. To a considerable extent, what the user sees is the file system. We will look at both the file-system interface and the file-system implementation.

Input/Output is covered in Chap. 5. We will cover the concept of device (in)dependence using examples such as storage devices, keyboards, and displays.

Chapter 6 is about deadlocks, including ways to prevent or avoid them.

At this point, we will have completed our study of the basic principles of single-CPU operating systems. However, there is more to say, especially about advanced topics. In Chap. 7, we examine virtualization. We discuss both the principles, and some of the existing virtualization solutions in detail. Another advanced topic is multiprocessor systems, including multicores, parallel computers, and distributed systems. These subjects are covered in Chap. 8. Another important subject is operating system security, which we cover in Chap 9.

Next we have some case studies of real operating systems. These are UNIX, Linux, and Android (Chap. 10), and Windows 11 (Chap. 11). The text concludes with some wisdom and thoughts about operating system design in Chap. 12.

1.11 METRIC UNITS

To avoid any confusion, it is worth stating explicitly that in this book, as in computer science in general, metric units are used instead of traditional English units (the furlong-stone-fortnight system). The principal metric prefixes are listed in Fig. 1-31. The prefixes are typically abbreviated by their first letters, with the units greater than 1 capitalized. Thus a 1-TB database occupies 10^{12} bytes of storage and a 100-psec (or 100-ps) clock ticks every 10^{-10} seconds. Since milli and micro both begin with the letter “m,” a choice had to be made. Normally, “m” is for milli and “ μ ” (the Greek letter mu) is for micro.

It is also worth pointing out that, in common industry practice, the units for measuring memory sizes have slightly different meanings. There kilo means 2^{10} (1024) rather than 10^3 (1000) because memories are always a power of two. Thus a 1-KB memory contains 1024 bytes, not 1000 bytes. Similarly, a 1-MB memory contains 2^{20} (1,048,576) bytes and a 1-GB memory contains 2^{30} (1,073,741,824) bytes. However, a 1-Kbps communication line transmits 1000 bits per second and a 1-Gbps LAN runs at 1,000,000,000 bits/sec because these speeds are not powers of two. Unfortunately, many people tend to mix up these two systems, especially for